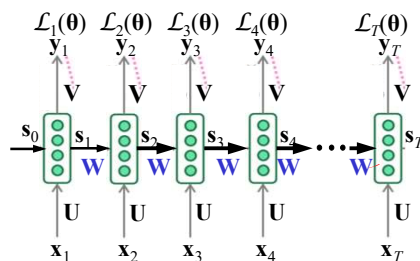


Neural Networks for Sequential Learning: Long Short Term Memory (LSTM)

1

Recurrent Neural Networks (RNNs)



$$\mathcal{L}(\theta) = \sum_{t=1}^T \mathcal{L}_t(\theta)$$

$$\nabla_v \mathcal{L}(\theta) = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{V}} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{V}}$$

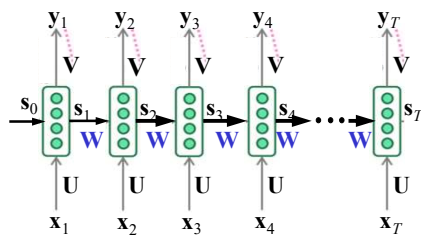
$$\nabla_u \mathcal{L}(\theta) = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{U}} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{U}}$$

$$\nabla_w \mathcal{L}(\theta) = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{W}} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{W}}$$

- RNN used for **sequential learning problem**
 - Each input is dependent on the previous or future input
 - The input is not of a fixed size
- RNN uses **backpropagation through time** for training
- Backpropagate the total loss and update the parameter set θ of the network
- For backpropagation we need to compute the **gradients** w.r.t. $\mathbf{U}, \mathbf{V}, \mathbf{W}, \mathbf{c}, \mathbf{b}$

2

Recurrent Neural Networks



- RNN used for sequential learning problem
 - Each input is dependent on the previous or future input
 - The input is not of a fixed size
- RNN uses backpropagation through time for training
- Backpropagate the total loss and update the parameter set θ of the network

$$\nabla_{\mathbf{w}} \mathcal{L}(\theta) = \frac{\partial \mathcal{L}(\theta)}{\partial \mathbf{w}} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{w}}$$

$\mathbf{w}$: weights in the recurrent connections •

$$\frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{w}} = \frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{s}_t} \left(\sum_{k=1}^t \frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k} \frac{\partial^+ \mathbf{s}_k}{\partial \mathbf{w}} \right)$$

$$\frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{w}} = \frac{\partial \mathcal{L}_t(\theta)}{\partial \mathbf{s}_t} \left(\sum_{k=1}^t \prod_{j=k+1}^t \frac{\partial \mathbf{s}_j}{\partial \mathbf{s}_{j-1}} \frac{\partial^+ \mathbf{s}_k}{\partial \mathbf{w}} \right)$$

- For backpropagation we need to compute the gradients w.r.t. $\mathbf{U}, \mathbf{V}, \mathbf{w}, \mathbf{c}, \mathbf{b}$

- RNN suffer from **vanishing/exploding gradients problem**

- Let us look into some concepts that help solving this problem
 - Long short term memory (LSTM) and gated recurrent unit (GRU)

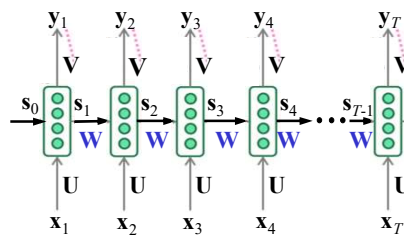
3

LSTM and GRU

- Both LSTM and GRU are the variants of RNN
 - They are just a better modeling choice
- First, let us introduce the notion of **selective read**, **selective write** and **selective forget** as a solution to address the issues in RNN
- Then, we will build on this intuition and see how we could realize it using LSTM and GRU

4

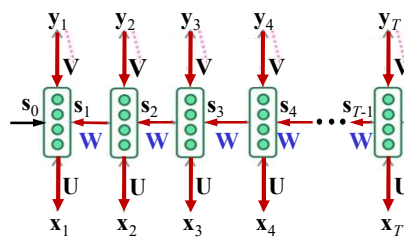
Issues with RNN



- The state (s_t) of an RNN records information from all previous time steps
- At each new timestep, s_t include the information at current timestep and also the accumulated history from previous timestep
- The state (s_t) can be seen as fixed size memory (M -dimensional vector)
- At each new timestep the old information gets morphed by the current input
- One could imagine that after t steps the information stored at time step $t-k$ (for some $k < t$) gets completely morphed so much that it would be impossible to extract the original information stored at time step $t-k$

5

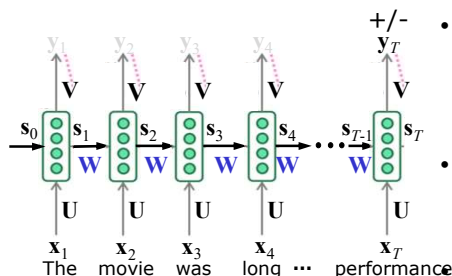
Issues with RNN



- A similar problem occurs when the information flows backwards (backpropagation)
- It is very hard to assign the responsibility of the error caused at time step t to the events that occurred at time step $t-k$
- This responsibility is of course in the form of gradients and we studied the problem in backward flow of gradients
 - Vanishing gradient problem

6

Issues with RNN



Review: The movie was long but the direction was excellent. The lead actor delivered an amazing performance.

Review: The direction was excellent. The lead actor delivered an amazing performance. However, the movie was long

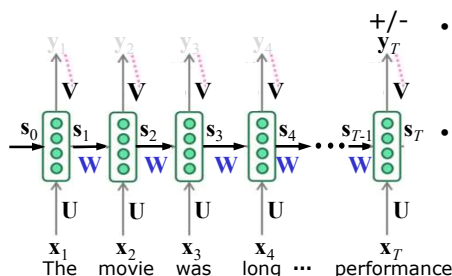
- Consider the task of **predicting the sentiment (positive/negative)** of a movie review
- RNN reads the document from left to right and after every word updates the state

By the time we reach the end of the document the information obtained from the first few words is completely lost

- Ideally we want to
 - selectively write** new information from the current word to the state
 - selectively read** the information added by previous sentiment bearing words (long, but, excellent, amazing, etc.)
 - selectively forget** the information added by stop words (was, the, etc.) and unimportant words

7

Issues with RNN



Review: The movie was long but the direction was excellent. The lead actor delivered an amazing performance.

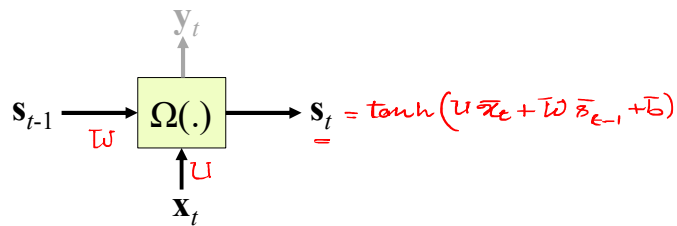
- How do we convert this intuition into mathematical equations?
- Recall that
 - the vector (s_t) is called the **state of the RNN**
 - It has a **finite size** ($s_t \in \mathbb{R}^M$) and is used to **store all the information** up to timestep t
- Sooner or later it will get overloaded

- The information from the initial states will get **morphed** beyond recognition
- We believe that **selective write**, **selective read** and **selective forget** to ensure that this **finite sized state vector is used effectively**

8

Selective Write, Selective Read and Selective Forget

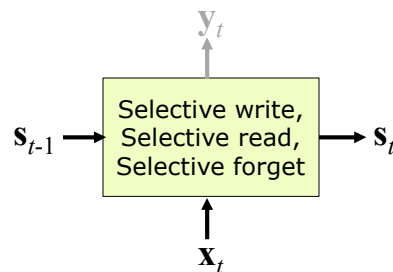
- We have computed a state s_{t-1} at timestep $t - 1$ and now we want to overload it with **new information** (x_t) and **compute a new state** (s_t)



9

Selective Write, Selective Read and Selective Forget

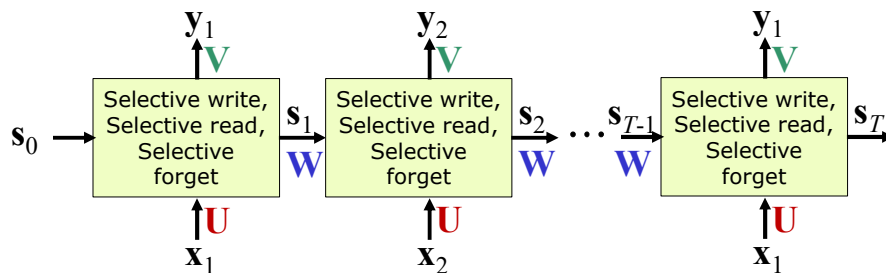
- We have computed a state s_{t-1} at timestep $t - 1$ and now we want to overload it with **new information** (x_t) and **compute a new state** (s_t)
- While going from s_{t-1} to s_t , **selective write**, **selective read** and **selective forget** will be used so that only **important information** is retained in s_t



10

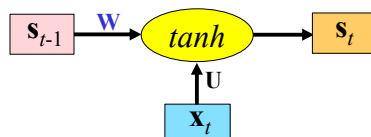
Selective Write, Selective Read and Selective Forget

- We have computed a state s_{t-1} at timestep $t - 1$ and now we want to overload it with **new information** (x_t) and **compute a new state** (s_t)
- While going from s_{t-1} to s_t , **selective write**, **selective read** and **selective forget** will be used so that only **important information is retained in s_t**



11

Selective Write



$$s_{t-1} = \begin{bmatrix} 0.8 \\ 0.6 \\ 1.9 \\ 2.6 \end{bmatrix} \odot \begin{bmatrix} 0.1 \\ 0.6 \\ 0.3 \\ 0.01 \end{bmatrix} = \begin{bmatrix} 0.08 \\ 0.36 \\ 0.57 \\ 0.26 \end{bmatrix}$$

- Recall that RNN uses s_{t-1} to compute s_t $s_t = \tanh(Ux_t + Ws_{t-1} + b)$
- But now instead of passing s_{t-1} as it is to compute s_t we want to **pass (write) only some portions of it to the next state**
- Multiply each entry of s_{t-1} with a **value between 0 and 1** which determines what **fraction of the current state to pass on to the next state**

12

Selective Write

$$\mathbf{s}_{t-1} \odot \mathbf{o}_{t-1} = \mathbf{h}_{t-1}$$

Selective write

 $\mathbf{s}_t$
 $\mathbf{x}_t$

- We introduce a output gate which is a vector $\mathbf{o}_{t-1}$ which decides what fraction of each element of $\mathbf{s}_{t-1}$ should be passed to the next state
 - Each element of $\mathbf{o}_{t-1}$ is restricted to be between 0 and 1
- Each element of $\mathbf{o}_{t-1}$ gets multiplied with the corresponding element of $\mathbf{s}_{t-1}$
 - It results into a vector $\mathbf{h}_{t-1}$
- $\mathbf{h}_{t-1}$ contain what fraction of $\mathbf{s}_{t-1}$ should be passed (written) to the next state
- How do we compute $\mathbf{o}_{t-1}$? How does the RNN know what fraction of the state to pass on?

13

Selective Write

$$\mathbf{s}_{t-1} \odot \mathbf{o}_{t-1} = \mathbf{h}_{t-1}$$

Selective write

 $\mathbf{s}_t$
 $\mathbf{x}_t$

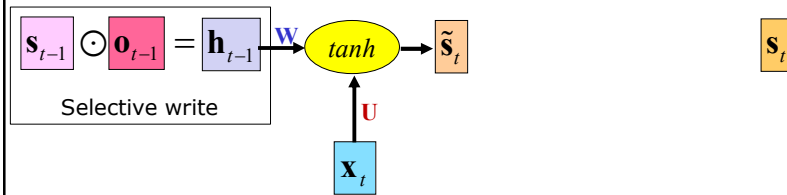
- We compute $\mathbf{o}_{t-1}$ and $\mathbf{h}_{t-1}$ as:

$$\mathbf{o}_{t-1} = \text{logistic}(\mathbf{U}^{(o)} \mathbf{x}_{t-1} + \mathbf{W}^{(o)} \mathbf{h}_{t-2} + \mathbf{b}^{(o)})$$

$$\mathbf{h}_{t-1} = \mathbf{o}_{t-1} \odot \tanh(\mathbf{s}_{t-1})$$
- The parameters $\mathbf{W}^{(o)}$, $\mathbf{U}^{(o)}$ and $\mathbf{b}^{(o)}$ need to be learned along with the existing parameters $\mathbf{W}$, $\mathbf{U}$, $\mathbf{V}$, $\mathbf{c}$, $\mathbf{b}$
- The sigmoid (logistic) function ensures that the values are between 0 and 1
- $\mathbf{o}_t$ is called the output gate as it decides how much to pass (write) to the next time step

14

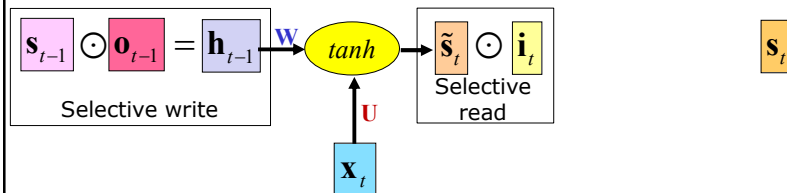
Selective Read



- We will now use $\mathbf{h}_{t-1}$ to compute the new state at the next time step
- We will also use $\mathbf{x}_t$ which is the new input at time step t
- $\tilde{\mathbf{s}}_t = \tanh(\mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{h}_{t-1} + \mathbf{b})$
- The parameters $\mathbf{U}$, $\mathbf{W}$ and $\mathbf{b}$ are similar to the parameters that are used in RNN
- $\tilde{\mathbf{s}}_t$ is an **intermediate state** which captures all the information from the previous state ($\mathbf{h}_{t-1}$) and the current input $\mathbf{x}_t$
- However, we are not using **all this new information**
- We only **selectively read from it** before constructing the new cell state $\mathbf{s}_t$

15

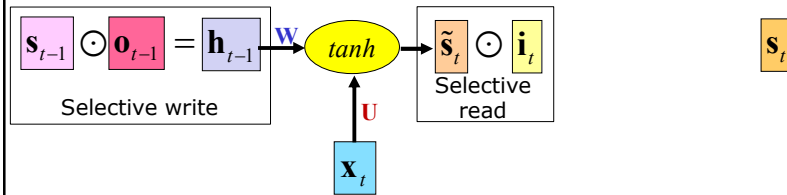
Selective Read



- We introduce another gate called the **input gate**
- $\mathbf{i}_t = \text{logistic}(\mathbf{U}^{(i)}\mathbf{x}_t + \mathbf{W}^{(i)}\mathbf{h}_{t-1} + \mathbf{b}^{(i)})$
- We use $\mathbf{i}_t \odot \tilde{\mathbf{s}}_t$ (element-wise product) as the **selectively read state information**

16

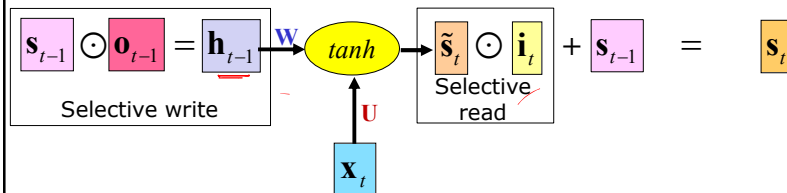
So far...



- So far we have the following:
- **Previous state:** $\mathbf{s}_{t-1}$
- **Output gate:** $\mathbf{o}_{t-1} = \text{logistic}(\mathbf{U}^{(o)}\mathbf{x}_{t-1} + \mathbf{W}^{(o)}\mathbf{h}_{t-2} + \mathbf{b}^{(o)})$
- **Selective write:** $\mathbf{h}_{t-1} = \mathbf{o}_{t-1} \odot \tanh(\mathbf{s}_{t-1})$
- **Current (temporary) state:** $\tilde{\mathbf{s}}_t = \tanh(\mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{h}_{t-1} + \mathbf{b})$
- **Input gate:** $\mathbf{i}_t = \text{logistic}(\mathbf{U}^{(i)}\mathbf{x}_t + \mathbf{W}^{(i)}\mathbf{h}_{t-1} + \mathbf{b}^{(i)})$
- **Selective read:** $\mathbf{i}_t \odot \tilde{\mathbf{s}}_t$

17

Selective Forget



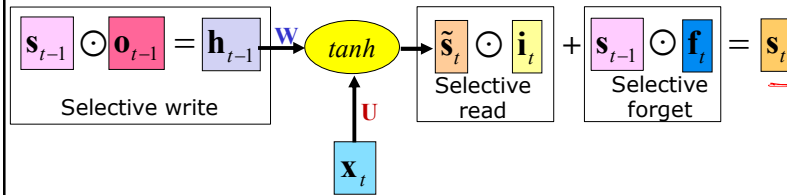
- How do we combine $\mathbf{h}_{t-1}$ and $\tilde{\mathbf{s}}_t$ to get the new state?
- Simple (but effective) way of combining:

$$\mathbf{s}_t = \mathbf{s}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{s}}_t$$
- But we may not want to use the whole of $\mathbf{s}_{t-1}$ but **forget some parts of it**
- We introduce the **forget gate**

$$\mathbf{f}_{t-1} = \text{logistic}(\mathbf{U}^{(f)}\mathbf{x}_t + \mathbf{W}^{(f)}\mathbf{h}_{t-1} + \mathbf{b}^{(f)})$$

18

Selective Forget



- How do we combine $\mathbf{h}_{t-1}$ and $\tilde{\mathbf{s}}_t$ to get the new state?
- Simple (but effective) way of combining:

$$\mathbf{s}_t = \mathbf{s}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{s}}_t$$
- But we may not want to use the whole of $\mathbf{s}_{t-1}$ but **forget some parts of it**
- We introduce the **forget gate**

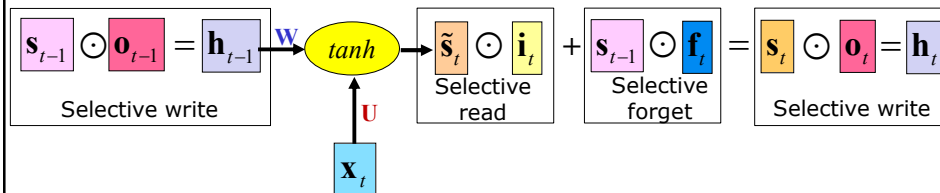
$$\mathbf{f}_t = \text{logistic}(\mathbf{U}^{(f)} \mathbf{x}_t + \mathbf{W}^{(f)} \mathbf{h}_{t-1} + \mathbf{b}^{(f)})$$

- The **new state** $\mathbf{s}_t$ is obtained as

$$\mathbf{s}_t = \underbrace{\mathbf{f}_t \odot \mathbf{s}_{t-1}} + \mathbf{i}_t \odot \tilde{\mathbf{s}}_t$$

19

Selective Forget



- How do we combine $\mathbf{h}_{t-1}$ and $\tilde{\mathbf{s}}_t$ to get the new state?
- Simple (but effective) way of combining:

$$\mathbf{s}_t = \mathbf{s}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{s}}_t$$
- But we may not want to use the whole of $\mathbf{s}_{t-1}$ but **forget some parts of it**
- We introduce the **forget gate**

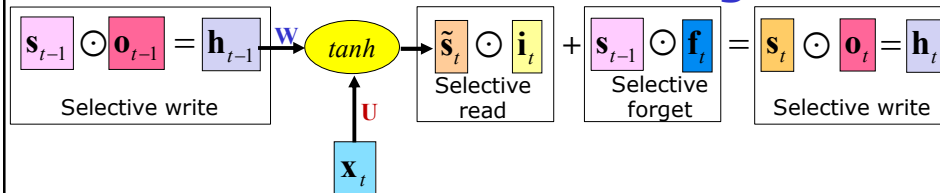
$$\mathbf{f}_{t-1} = \text{logistic}(\mathbf{U}^{(f)} \mathbf{x}_t + \mathbf{W}^{(f)} \mathbf{h}_{t-1} + \mathbf{b}^{(f)})$$

- The **new state** $\mathbf{s}_t$ is obtained as

$$\mathbf{s}_t = \mathbf{f}_t \odot \mathbf{s}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{s}}_t$$

20

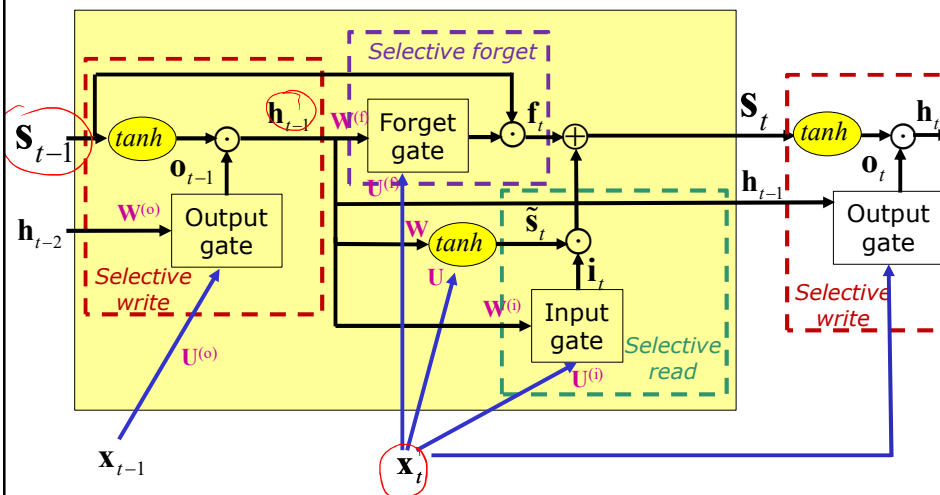
LSTM Cell: Selective Write, Selective Read and Selective Forget



- Previous state: s_{t-1}
- Output gate: $o_{t-1} = \text{logistic}(U^{(o)}x_{t-1} + W^{(o)}h_{t-2} + b^{(o)})$
- Selective write: $h_{t-1} = o_{t-1} \odot \tanh(s_{t-1})$
- Current (temporary) state: $\tilde{s}_t = \tanh(Ux_t + Wh_{t-1} + b)$
- Input gate: $i_t = \text{logistic}(U^{(i)}x_t + W^{(i)}h_{t-1} + b^{(i)})$
- Selective read: $i_t \odot \tilde{s}_t$
- Forget gate: $f_t = \text{logistic}(U^{(f)}x_t + W^{(f)}h_{t-1} + b^{(f)})$
- Selective forget: $f_t \odot s_{t-1}$
- Current (final) state: $s_t = (f_t \odot s_{t-1}) + (i_t \odot \tilde{s}_t)$

21

LSTM Cell



22

LSTM

- LSTM has many variants which include different number of gates and also different arrangement of gates
- The one which we just saw is one of the most popular variants of LSTM
- Another equally popular variant of LSTM is gated recurrent unit (GRU)
- The different variants of LSTM architecture has been discussed in the article "LSTM: A Search Space Odyssey" [1] *reading assignment.*

[1] Klaus Greff, Rupesh Kumar Srivastava, Jan Koutník, Bas R. Steunebrink, and Jürgen Schmidhuber, "LSTM: A Search Space Odyssey," in IEEE Transactions on Neural Networks and Learning Systems, vol. 20, issue 10, pp. 2222 - 2232, 2017

23

Gated Recurrent Unit (GRU)

- **Gates:**
 - **Output gate:** $\mathbf{o}_t = \text{logistic}(\mathbf{U}^{(o)}\mathbf{x}_t + \mathbf{W}^{(o)}\mathbf{s}_{t-1} + \mathbf{b}^{(o)})$
 - **Input gate:** $\mathbf{i}_t = \text{logistic}(\mathbf{U}^{(i)}\mathbf{x}_t + \mathbf{W}^{(i)}\mathbf{s}_{t-1} + \mathbf{b}^{(i)})$
- **States:**
 - **Current (temporary) state:** $\tilde{\mathbf{s}}_t = \tanh(\mathbf{U}\mathbf{x}_t + \mathbf{W}(\mathbf{o}_{t-1} \odot \mathbf{s}_{t-1}) + \mathbf{b})$
 - **New state:** $\mathbf{s}_t = ((\mathbf{1} - \mathbf{i}_t) \odot \mathbf{s}_{t-1}) + (\mathbf{i}_t \odot \tilde{\mathbf{s}}_t)$
- **No explicit forget gate** (the forget gate and input gates are tied)
- The gates depend directly on $\mathbf{s}_{t-1}$ and not the intermediate $\mathbf{h}_{t-1}$ as in the case of LSTMs

24

LSTMs Avoid the Problem of Vanishing Gradients

- Intuition:
 - During forward propagation the gates control the flow of information
 - They prevent any irrelevant information from being written to the state
 - Similarly during backward propagation they control the flow of gradients
 - It is easy to see that during backward pass the gradients will get multiplied by the gate
 - We know that, $\mathbf{s}_t = (\mathbf{f}_t \odot \mathbf{s}_{t-1}) + (\mathbf{i}_t \odot \tilde{\mathbf{s}}_t)$ --- (1)
 - We also know that,

$$\nabla_{\mathbf{w}} \mathcal{L}(\boldsymbol{\theta}) = \frac{\partial \mathcal{L}(\boldsymbol{\theta})}{\partial \mathbf{W}} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{W}} \quad \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{s}_t} \left(\sum_{k=1}^t \prod_{j=k+1}^t \frac{\partial \mathbf{s}_j}{\partial \mathbf{s}_{j-1}} \frac{\partial^+ \mathbf{s}_k}{\partial \mathbf{W}} \right)$$
 - While computing the gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{s}_{j-1}}$ if the second term in (1) vanishes (at the worst case), we still have the first term in the gradient

25

LSTMs Avoid the Problem of Vanishing Gradients

- Intuition:
 - During forward propagation the gates control the flow of information
 - They prevent any irrelevant information from being written to the state
 - Similarly during backward propagation they control the flow of gradients
 - It is easy to see that during backward pass the gradients will get multiplied by the gate
 - We know that, $\mathbf{s}_t = (\mathbf{f}_t \odot \mathbf{s}_{t-1}) + (\mathbf{i}_t \odot \tilde{\mathbf{s}}_t)$ --- (1) $\|\tilde{\mathbf{f}}_t\| \rightarrow 0$
 - We also know that,

$$\nabla_{\mathbf{w}} \mathcal{L}(\boldsymbol{\theta}) = \frac{\partial \mathcal{L}(\boldsymbol{\theta})}{\partial \mathbf{W}} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{W}} \quad \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}_t(\boldsymbol{\theta})}{\partial \mathbf{s}_t} \left(\sum_{k=1}^t \prod_{j=k+1}^t \frac{\partial \mathbf{s}_j}{\partial \mathbf{s}_{j-1}} \frac{\partial^+ \mathbf{s}_k}{\partial \mathbf{W}} \right)$$
 - If the state at time $t-1$ did not contribute much to the state at time t then during backpropagation the gradients flowing into $\mathbf{s}_{t-1}$ will vanish
 - But this kind of a vanishing gradient is fine (since $\mathbf{s}_{t-1}$ did not contribute to $\mathbf{s}_t$ we don't want to hold it responsible for the crimes of $\mathbf{s}_t$)

26

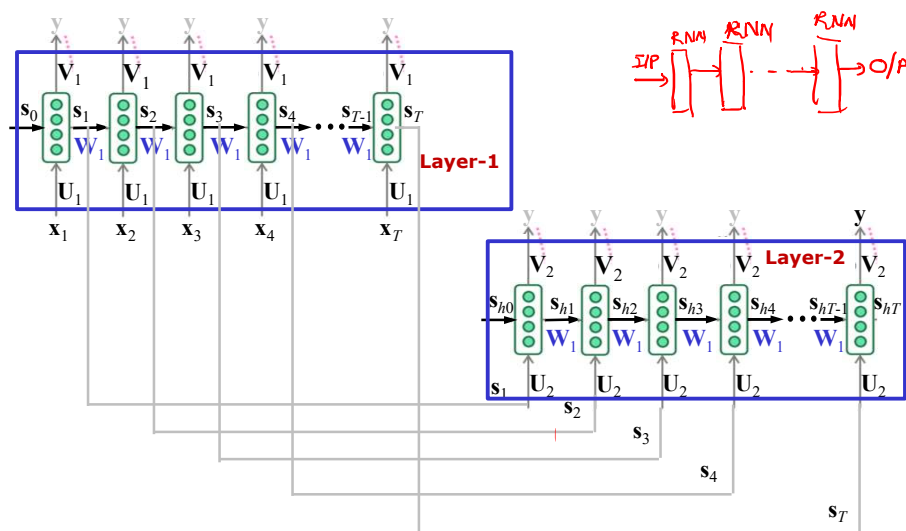
LSTMs Avoid the Problem of Vanishing Gradients

- Intuition:
 - During forward propagation the gates control the flow of information
 - They prevent any irrelevant information from being written to the state
 - Similarly during backward propagation they control the flow of gradients
 - It is easy to see that during backward pass the gradients will get multiplied by the gate
 - If the state at time $t-1$ did not contribute much to the state at time t then during backpropagation the gradients flowing into s_{t-1} will vanish
 - But this kind of a vanishing gradient is fine (since s_{t-1} did not contribute to s_t , we don't want to hold it responsible for the crimes of s_t)
- The key difference from vanilla RNNs is that the flow of information and gradients is controlled by the gates which ensure that the gradients vanish only when they should (i.e., when s_{t-1} didn't contribute much to s_t)

27

Two or More Layer RNN

- Task: Classification



28

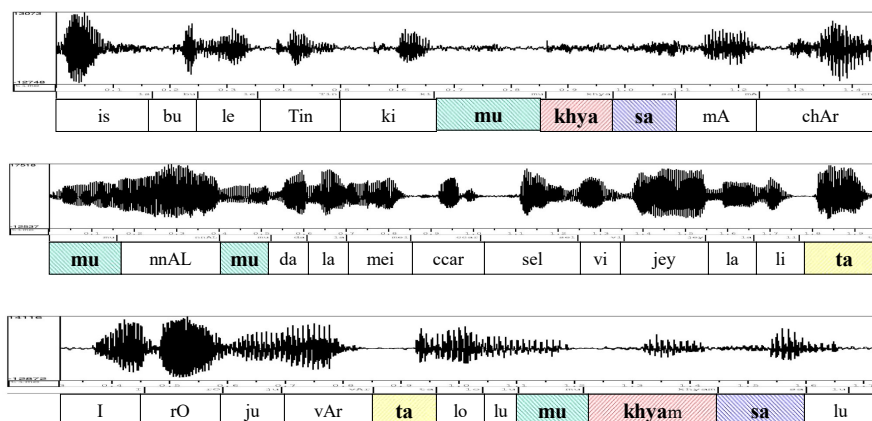
Summary

- RNN has the issue of vanishing gradients while modeling the sequential learning
- LSTM and GRU addresses the issue of vanishing gradient by **introducing gates** and performing **selective write, selective read** and **selective forget**
- LSTM and GRU are the **variants of RNN**
 - They are just the **better modelling choice** with additional parameters

29

Speech Processing Tasks

Syllables.

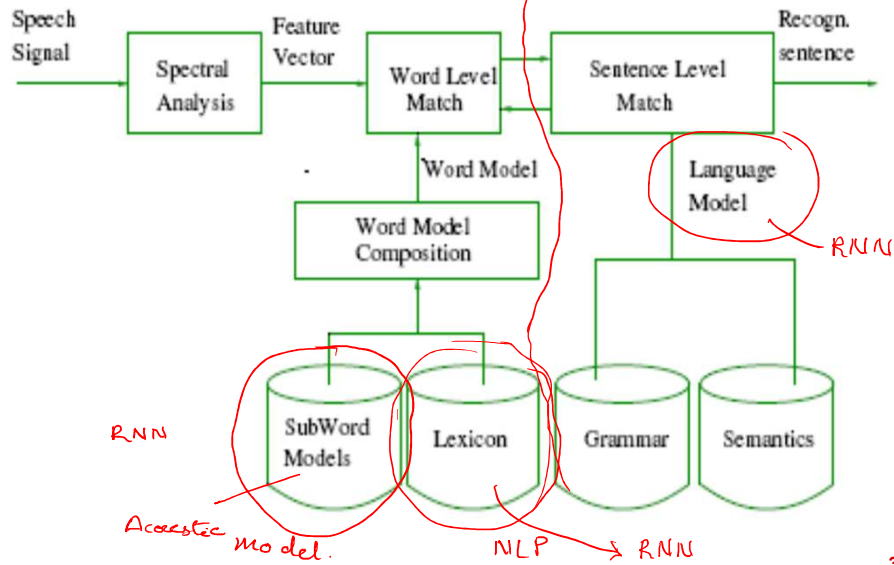


Speech Recognition
Speaker Verification/Recognition
Spoken Language Identification

Speech Emotion Recognition
Speech-to-speech Translation

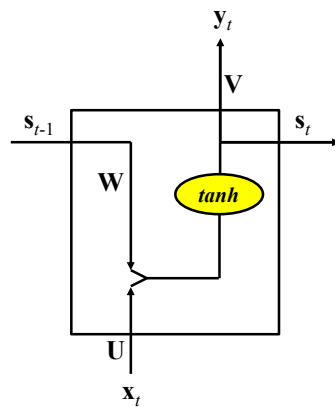
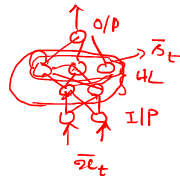
30

Continuous Speech Recognition

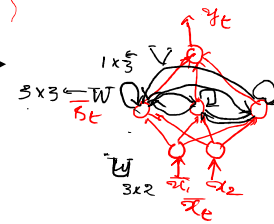


31

RNN Cell



HA PA
ni pa
9I = n i



$$\underline{s}_t = \tanh(Ux_t + Ws_{t-1} + b)$$

$$\underline{y}_t = \text{softmax}(Vs_t + c)$$

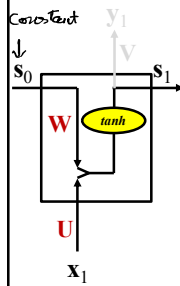
$\text{softmax}(\cdot)$ is when RNN is used as classifier

32

RNN

- Consider the task of predicting the sentiment (positive/negative) of a movie review

– Example: Movie is long, but it is amazing <stop> $\vec{y}_t = \begin{bmatrix} 0 & 1 \end{bmatrix}$



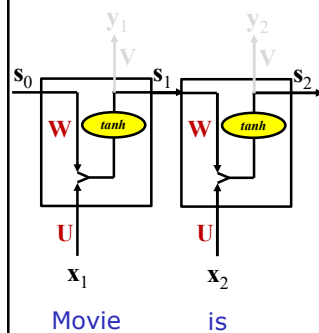
$$s_1 = \tanh(Ux_1 + Ws_0 + b)$$

33

RNN

- Consider the task of predicting the sentiment (positive/negative) of a movie review

– Example: Movie is long, but it is amazing <stop>

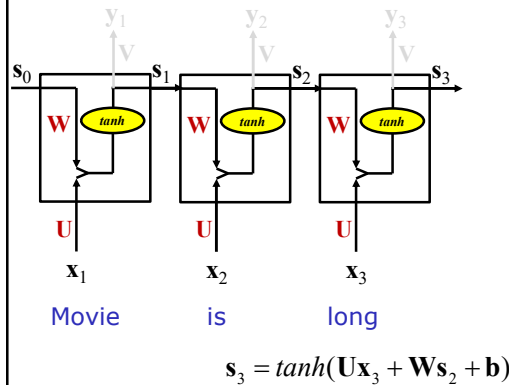


$$s_2 = \tanh(Ux_2 + Ws_1 + b)$$

34

RNN

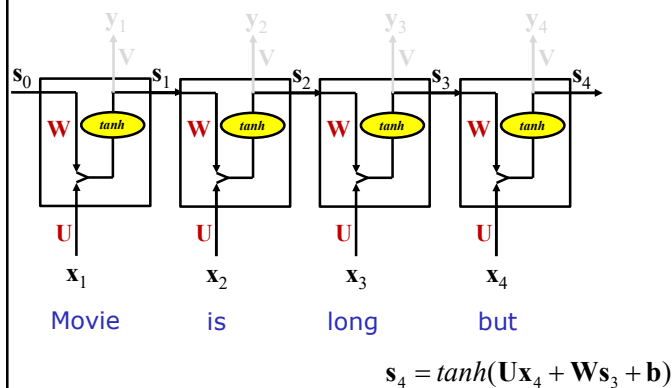
- Consider the task of predicting the sentiment (positive/negative) of a movie review
 - Example: Movie is long, but it is amazing <stop>



35

RNN

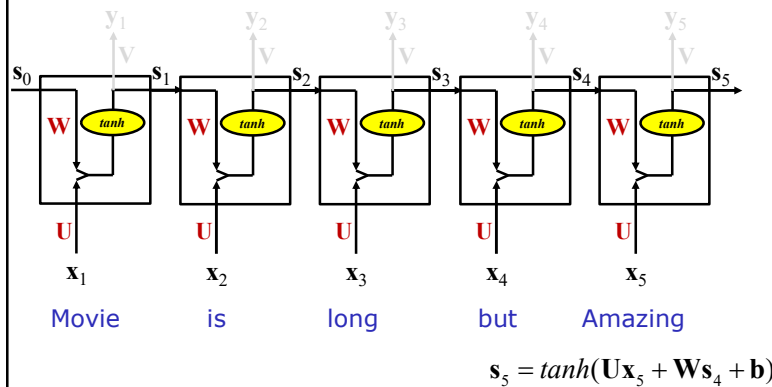
- Consider the task of predicting the sentiment (positive/negative) of a movie review
 - Example: Movie is long, but it is amazing <stop>



36

RNN

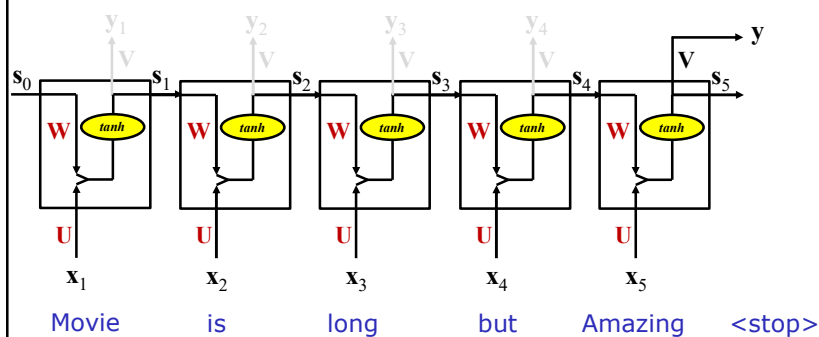
- Consider the task of predicting the sentiment (positive/negative) of a movie review
 - Example: Movie is long, but it is amazing <stop>



37

RNN

- Consider the task of predicting the sentiment (positive/negative) of a movie review
 - Example: Movie is long, but it is amazing <stop>



$$y = \text{softmax}(Vs_5 + c)$$

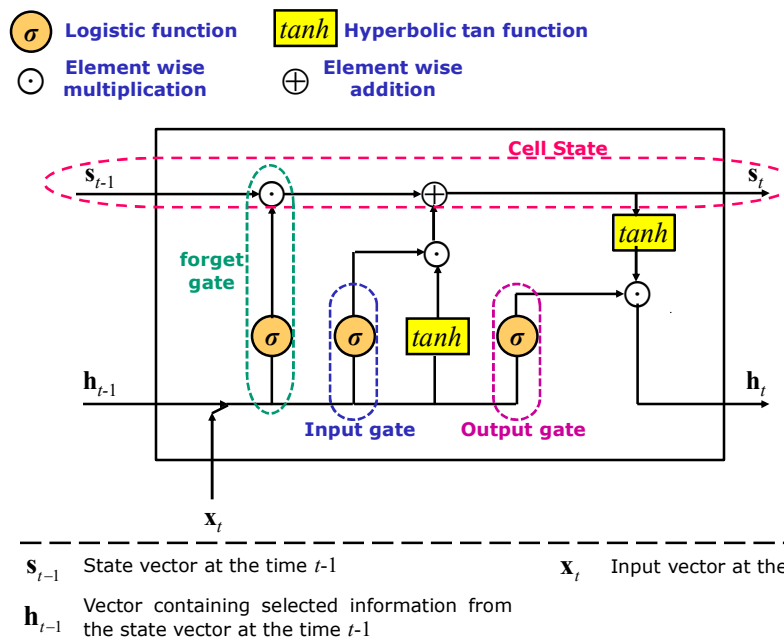
38

LSTM

- Consider the task of predicting the sentiment (positive/negative) of a movie review
 - Example: Movie is long, but amazing
- Selective forget: forget gate
 - forget the information added by unimportant words
 - Example: Movie is long, but amazing
- Selective read: input gate
 - read new information from the current word to the state
- Selective write: output gate
 - write the information added by previous sentiment bearing words i.e., pass the selected information to the next state

39

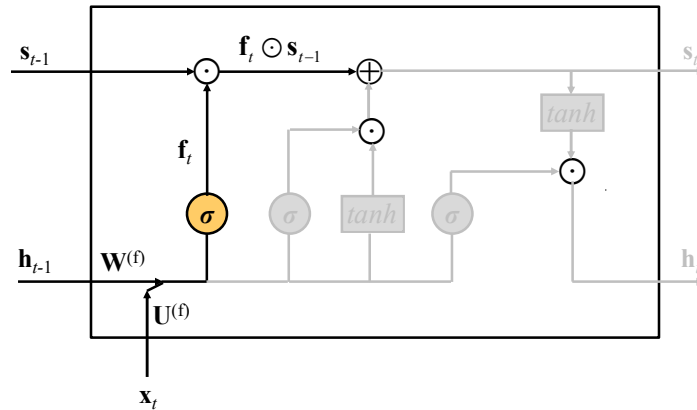
LSTM Cell



40

LSTM Cell

- **Selective forget: forget gate**
 - forget the information added by unimportant input

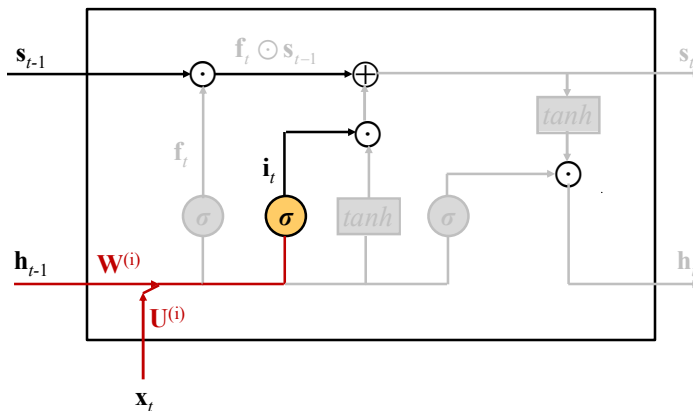


$$\mathbf{f}_t = \text{logistic}(\mathbf{U}^{(f)}\mathbf{x}_t + \mathbf{W}^{(f)}\mathbf{h}_{t-1} + \mathbf{b}^{(f)})$$

41

LSTM Cell

- **Selective read: input gate**
 - read new information from the current input to the state

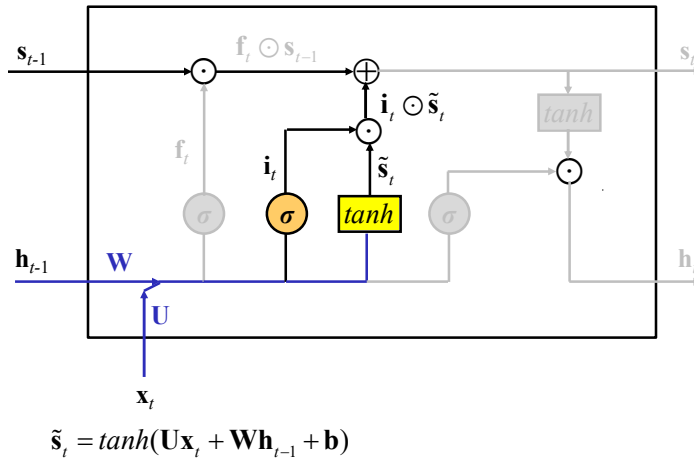


$$\mathbf{i}_t = \text{logistic}(\mathbf{U}^{(i)}\mathbf{x}_t + \mathbf{W}^{(i)}\mathbf{h}_{t-1} + \mathbf{b}^{(i)})$$

42

LSTM Cell

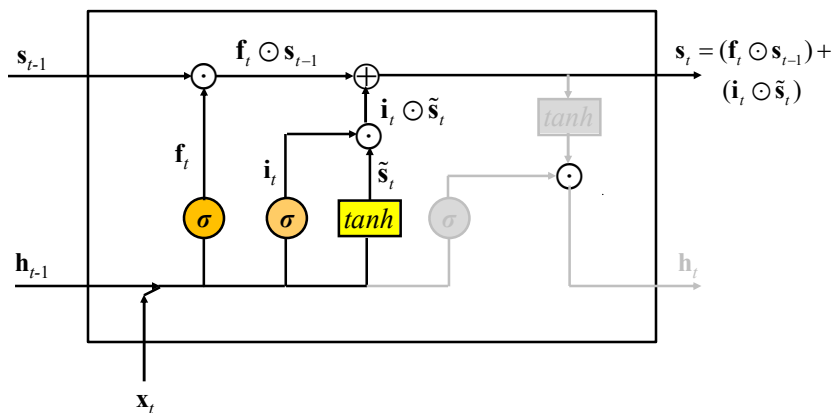
- Selective read: input gate
 - read new information from the current input to the state



43

LSTM Cell

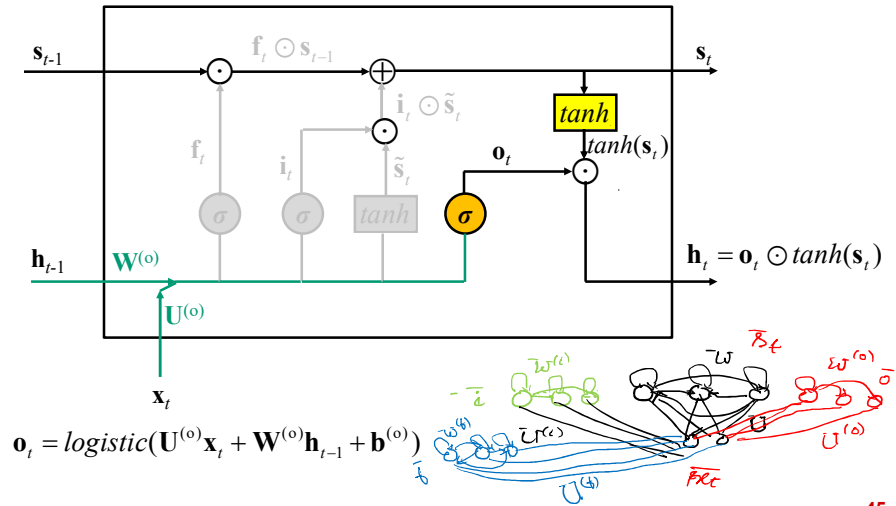
- New state generation



44

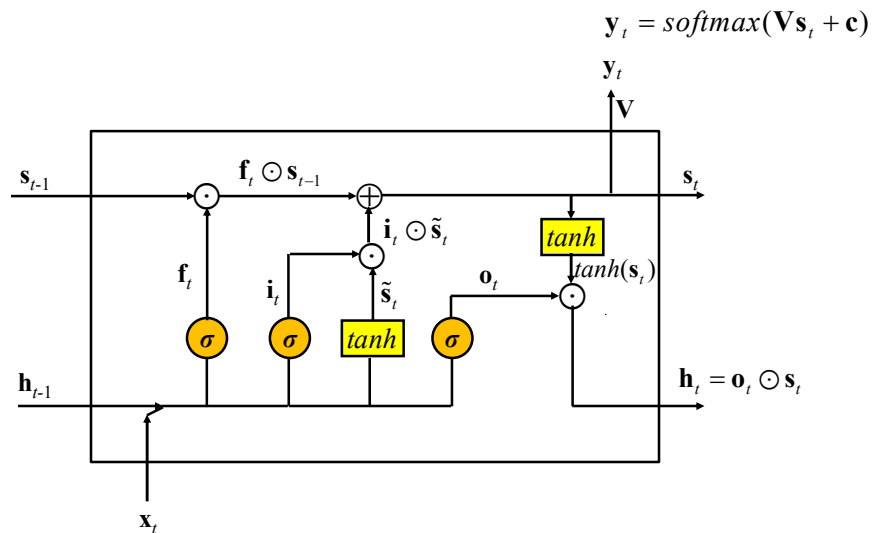
LSTM Cell

- **Selective write: output gate**
 - Pass/write the selected information to the next state



45

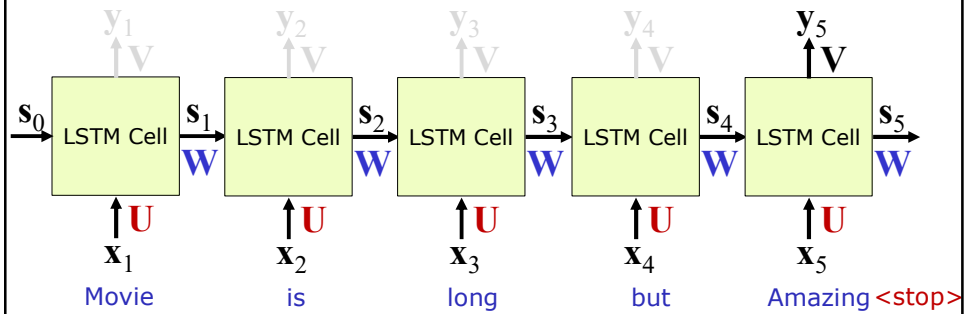
LSTM Cell



46

LSTM

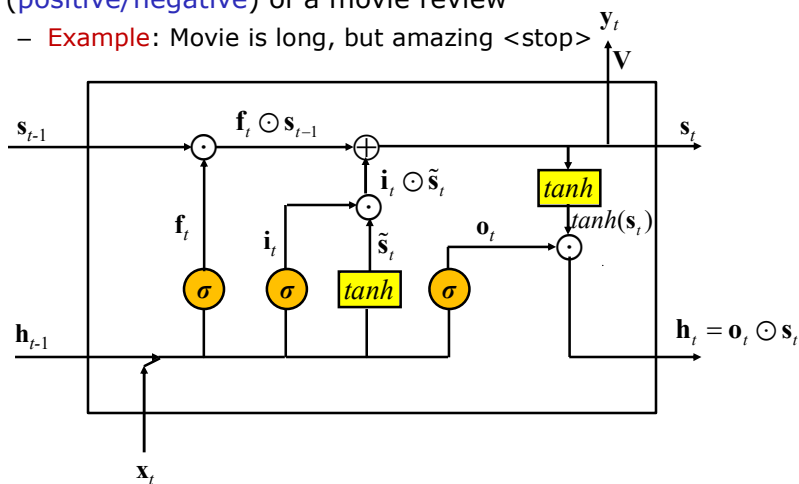
- Consider the task of predicting the sentiment (positive/negative) of a movie review
 - Example:** Movie is long, but amazing <stop>



47

LSTM

- Consider the task of predicting the sentiment (positive/negative) of a movie review
 - Example:** Movie is long, but amazing <stop>

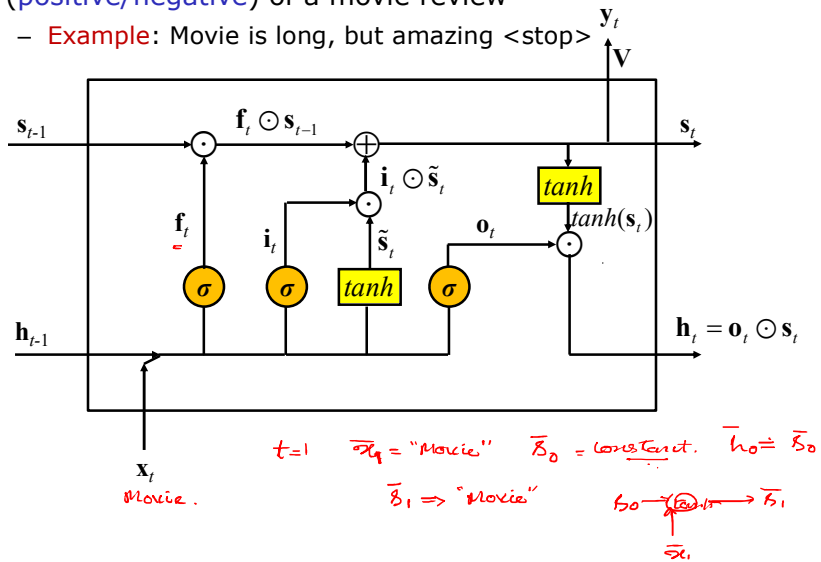


48

LSTM

- Consider the task of predicting the sentiment (positive/negative) of a movie review

– Example: Movie is long, but amazing <stop>

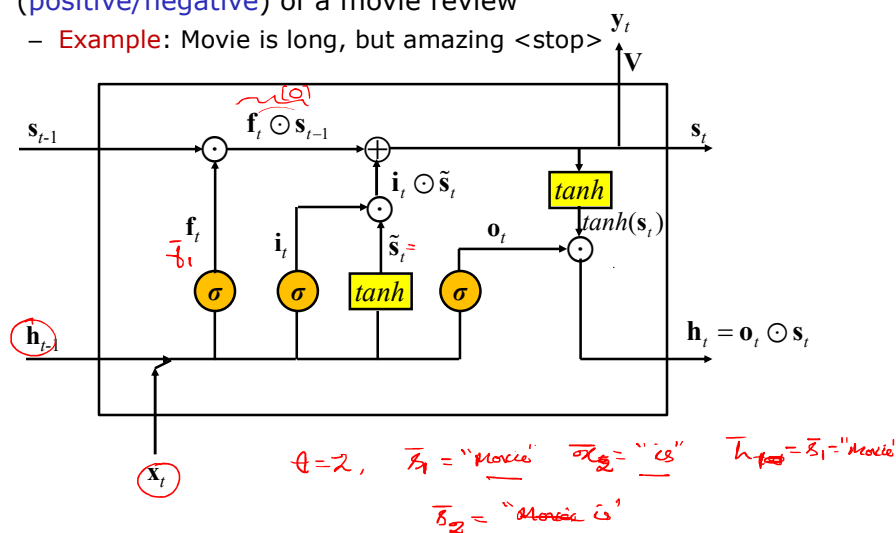


49

LSTM

- Consider the task of predicting the sentiment (positive/negative) of a movie review

– Example: Movie is long, but amazing <stop>



50

Text Books

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, [Deep learning](#), MIT Press, Available online: <http://www.deeplearningbook.org>, 2016
2. Charu C. Aggarwal, [Neural Networks and Deep Learning](#), Springer, 2018
3. B. Yegnanarayana, [Artificial Neural Networks](#), Prentice-Hall of India, 1999.
4. Satish Kumar, [Neural Networks - A Class Room Approach](#), Second Edition, Tata McGraw-Hill, 2013.
5. S. Haykin, [Neural Networks and Learning Machines](#), Prentice Hall of India, 2010.
6. C. M. Bishop, [Pattern Recognition and Machine Learning](#), Springer, 2006.
7. J. Han and M. Kamber, [Data Mining: Concepts and Techniques](#), Third Edition, Morgan Kaufmann Publishers, 2011.
8. S. Theodoridis and K. Koutroumbas, [Pattern Recognition](#), Academic Press, 2009.

51